Práctica 2 - Multiplicación Rápida de Enteros: Algoritmo de Karatsuba

Análisis y Diseño de Algoritmos — Ingeniería de Sistemas (Universidad EAFIT)

Miguel Ángel Alzate (Líder Técnico) • Joseph Buenaños (Aritmética) • Samuel Quintero (Pruebas)

31 Octubre 2026

1. Introducción y Deducción de la Identidad de Karatsuba

La multiplicación clásica de dos números enteros de *n* dígitos requiere multiplicar cada dígito del primer operando por cada dígito del segundo, lo que conlleva una complejidad temporal asintótica de **Θ(n²)**. En 1960, Anatolii Karatsuba demostró que es posible romper esta barrera cuadrática aplicando el paradigma de *Divide y Vencerás* apoyado en una identidad algebraica que reduce el número de multiplicaciones recursivas de cuatro a tres.

Dados dos números enteros no negativos x e y de longitud n en base $B = 10$, ambos se descomponen en dos mitades de tamaño $m = n / 2$ (mitades altas x_1, y_1 y mitades bajas x_0, y_0):

$$x = x_1 \cdot 10^m + x_0 \quad ; \quad y = y_1 \cdot 10^m + y_0$$

Al expandir algebraicamente el producto $x \cdot y$, el enfoque directo agrupa la expresión en cuatro productos de números de $n / 2$ dígitos, lo que generaría la recurrencia $T(n) = 4T(n/2) + \Theta(n) = \Theta(n^2)$:

$$x \cdot y = (x_1 \cdot y_1) \cdot 10^{2m} + (x_1 \cdot y_0 + x_0 \cdot y_1) \cdot 10^m + (x_0 \cdot y_0) \quad (1)$$

La identidad de Karatsuba evita evaluar x_1y_0 y x_0y_1 por separado definiendo únicamente tres productos recursivos:

$$P_1 = x_1 \cdot y_1 \quad ; \quad P_2 = x_0 \cdot y_0 \quad ; \quad P_3 = (x_1 + x_0) \cdot (y_1 + y_0) = x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0$$

Sustrayendo P_1 y P_2 del producto combinado P_3, se despeja de forma exacta el término cruzado central a cambio de sumas y restas lineales:

$$P_3 - P_1 - P_2 = (x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0) - x_1y_1 - x_0y_0 = x_1y_0 + x_0y_1 \quad (2)$$

Reemplazando (2) en (1), el producto total se recompone con solo tres llamadas recursivas y desplazamientos en base 10:

$$x \cdot y = P_1 \cdot 10^{2m} + (P_3 - P_1 - P_2) \cdot 10^m + P_2 \quad (3)$$

2. Análisis Teórico y Resolución de la Recurrencia

Al reducir las multiplicaciones recursivas de 4 a 3 para subproblemas de tamaño $n / 2$, y dado que las sumas, restas y desplazamientos de cadenas toman tiempo lineal $\Theta(n)$ (incluso cuando $x_1 + x_0$ genera un dígito adicional $m + 1$), la ecuación de recurrencia queda expresada como:

$$T(n) = 3T(n / 2) + \Theta(n) \quad (4)$$

2.1 Resolución mediante el Teorema Maestro

Para una recurrencia de la forma $T(n) = aT(n / b) + f(n)$, identificamos los parámetros: **a = 3** (subproblemas recursivos), **b = 2** (factor de partición) y **f(n) = Θ(n)** (trabajo de partición y

recomposición). Calculamos la función crítica de las hojas: $n^{(\log_b a)} = n^{(\log_2 3)} \approx n^{1.585}$. Al comparar $f(n) = \Theta(n^1)$ frente a $n^{(\log_2 3)}$, verificamos que $f(n) = O(n^{(\log_2 3 - \varepsilon)})$ tomando cualquier constante $0 < \varepsilon \leq 0.58$. Esto ubica la recurrencia en el **Caso 1 del Teorema Maestro**, donde el peso computacional está polinómicamente dominado por las hojas:

$$T(n) = \Theta(n^{(\log_2 3)}) \approx \Theta(n^{1.585}) \quad (5)$$

2.2 Resolución mediante el Árbol de Recurrencia

El árbol de recurrencia permite deducir constructivamente el origen del exponente $\log_2 3$ analizando el trabajo nivel por nivel:

- **Nivel 0 (raíz):** 1 problema de tamaño n. Costo lineal: c · n.
- **Nivel 1:** 3 subproblemas de tamaño n / 2. Costo agregado: 3 · c(n / 2) = c · n · (3 / 2).
- **Nivel 2:** 9 subproblemas de tamaño n / 4. Costo agregado: 9 · c(n / 4) = c · n · (3 / 2)².
- **Nivel i:** 3^i subproblemas de tamaño n / 2^i. Costo en el nivel i: c · n · $(3 / 2)^i$.

La recursión alcanza el caso base cuando el tamaño del subproblema se reduce a 1 ($n / 2^h = 1 \Rightarrow h = \log_2 n$). En el nivel de las hojas ($i = \log_2 n$), existen exactamente $3^{(\log_2 n)}$ subproblemas triviales. Por la identidad logarítmica de cambio de base ($a^{(\log_b c)} = c^{(\log_b a)}$), se obtiene que **$3^{(\log_2 n)} = n^{(\log_2 3)}$**. Al sumar el trabajo de todos los niveles:

$$T(n) = \sum_{i=0}^{\log_2 n - 1} c \cdot n \cdot (3 / 2)^i + \Theta(n^{(\log_2 3)}) = c \cdot n \cdot [((3/2)^{(\log_2 n)} - 1) / (3/2 - 1)] + \Theta(n^{(\log_2 3)})$$

Dado que la razón de la serie geométrica es $r = 3 / 2 > 1$, la suma crece exponencialmente hacia la base del árbol y es asintóticamente proporcional a su último término: $n \cdot (3 / 2)^{(\log_2 n)} = n \cdot (n^{(\log_2 3)} / n) = n^{(\log_2 3)}$. Se concluye así que **$T(n) = \Theta(n^{(\log_2 3)}) \approx \Theta(n^{1.585})$**.

3. Decisiones de Diseño y Dificultades Encontradas

Representación y estructura modular: Los enteros de precisión arbitraria se representaron mediante *std::string* en base 10, separando el proyecto en *include/karatsuba.h* (contrato de interfaz), *src/operaciones.cpp* (suma, resta, desplazamiento y multiplicación escolar), *src/karatsuba.cpp* (normalización, partición y algoritmo recursivo) y *src/main_pruebas.cpp* (oráculo de validación modular mediante esquema de Horner con dos primos y benchmark).

Orden estricto de la resta sin aritmética de signos: Siguiendo la Sección 3 de la guía, la función *restar(A, B)* exige la precondición $A \geq B$. Para evitar resultados negativos intermedios, el término central se evaluó estrictamente en el orden **restar(restar(P_3, P_1), P_2)**. Como $P_3 - P_1 = P_2 + x_1y_0 + x_0y_1 \geq P_2 \geq 0$, ambas sustracciones garantizan matemáticamente operandos no negativos.

Hallazgo técnico crítico — La trampa del relleno a potencia de 2 en la recursión: Durante las primeras pruebas de rendimiento, el equipo detectó una anomalía severa: aunque Karatsuba aprobaba los 199 casos de correctitud, al duplicar *n* de 2048 a 4096 el tiempo se multiplicaba por **4.26×** (peor que el *3.98×* del clásico). Al instrumentar el código, descubrimos que invocar *rellenarPotenciaDeDos()* dentro de cada llamada recursiva destruía la recurrencia: cuando la suma $x_1 + x_0$ producía un acarreo de $m + 1$ dígitos, el relleno lo inflaba hasta la siguiente potencia de 2, devolviendo el subproblema P_3 al tamaño original **2m**

= n (generando 13.4 millones de llamadas para *n = 4096* en lugar de ~800 mil). **Solución:** Se desacopló la función de entrada *karatsuba()* —que rellena a potencia de 2 una única vez— de la función interna *karatsubaRec()*, la cual opera sobre las longitudes reales y aplica un umbral base *UMBRAL = 32* (configurable a 1 con *-DUMBRAL=1*), restaurando de inmediato la cota $\Theta(n^{1.585})$.

4. Análisis Experimental

El protocolo experimental se ejecutó compilando con *g++ -std=c++17 -O2* y midiendo con *std::chrono::steady_clock*. Para cada tamaño $n \in \{2, 4, ..., 4096\}$, el programa calibró dinámicamente el número de repeticiones hasta superar **60 ms** por medición, promediando 5 ensayos independientes y acumulando los resultados en un sumidero *volatile* para impedir optimizaciones de código muerto. Previamente se superaron los 199 casos de validación (1 dígito, ceros a la izquierda, ceros intercalados, longitudes impares y números grandes) con **0 errores**.

Dígitos (n)	Repeticiones	Clásica (µs)	Karatsuba (µs)	Ratio (Clásica / Kara)	Más Rápido
2	262,144	0.124	0.226	0.55×	Clásica
4	262,144	0.184	0.316	0.58×	Clásica
8	131,072	0.448	0.575	0.78×	Clásica
16	32,768	1.585	1.892	0.84×	Clásica
32	16,384	5.515	5.792	0.95×	Clásica
64	2,048	19.596	29.129	0.67×	Clásica
128	1,024	69.358	92.852	0.75×	Clásica
256	256	267.976	366.085	0.73×	Clásica
512	64	1,082.234	1,110.767	0.97×	Clásica (empate técnico)
1024	16	4,305.545	**3,385.972**	**1.27×**	**Karatsuba (Cruce)**
2048	4	17,159.038	**10,767.370**	**1.59×**	**Karatsuba**
4096	1	70,996.775	**31,866.995**	**2.23×**	**Karatsuba**

Cuadro 1: Tiempos de ejecución experimentales (en microsegundos) y razón de aceleración Clásica / Karatsuba.

Interpretación del punto de cruce y constantes ocultas: Los resultados demuestran un punto de cruce nítido en **n = 1024 dígitos** (ubicándose el cruce exacto entre 512 y 1024). Para entradas pequeñas ($n \leq 512$), la multiplicación clásica es más rápida porque opera sobre un único vector contiguo preasignado (*std::vector<int>*) mediante un doble bucle compacto con excelente localidad de caché. En contraste, Karatsuba incurre en constantes ocultas elevadas por nivel: 3 invocaciones recursivas, creación de subcadenas dinámicas (*substr*), llamadas a *sumar*, *restar* y *desplazar*, y normalización de ceros. Sin embargo, desde *n = 1024* la mejora asintótica domina de forma sostenida: en *n = 4096* Karatsuba es **2.23 veces más veloz**. Además, al duplicar *n* de 2048 a 4096, el tiempo clásico creció **4.14×** (consistente con

$2^2 = 4$ de $\Theta(n^2)$), mientras que Karatsuba creció apenas **2.96×**, confirmando empíricamente el factor teórico **2^(1.585) = 3**.

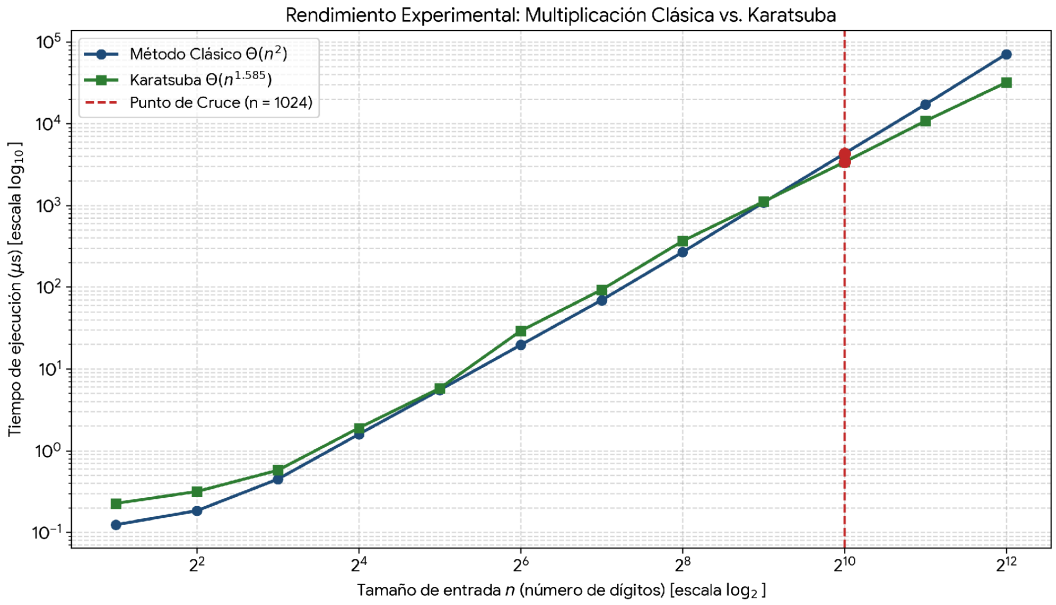


Figura 1: Comparación experimental en escala log-log del tiempo de ejecución frente al tamaño n, destacando el cruce en n = 1024.

5. Conclusiones

- **Validación teórico-práctica:** El Teorema Maestro y el árbol de recurrencia predicen con exactitud que duplicar el tamaño de entrada triplica el tiempo de ejecución en Karatsuba (2.96× experimental frente a 3× teórico), superando el crecimiento cuádruple (4.14×) del algoritmo escolar.
- **Impacto de las constantes ocultas:** Una menor complejidad asintótica no implica mayor velocidad para todo n; la sobrecarga de gestión de memoria dinámica en std::string hace que el método clásico sea preferible por debajo de ~512 dígitos y justifica usar un caso base híbrido (UMBRAL = 32).
- **Sensibilidad de la recurrencia al tamaño de los subproblemas:** El proyecto demostró que un detalle de implementación aparentemente inocuo —como rellenar a potencia de 2 dentro de la recursión— puede alterar el tamaño de P_3 de n/2 a n y destruir por completo la ventaja asintótica de Divide y Vencerás.

6. Declaración de Uso Ético de Inteligencia Artificial

En cumplimiento de la Sección 7 de la guía, el equipo declara el uso de herramientas de IA Generativa (Claude y Gemini) exclusivamente como apoyo al aprendizaje y depuración técnica: (1) configuración del Makefile y organización modular del repositorio, (2) diagnóstico de la degradación de complejidad ocasionada por el relleno intermedio a potencias de 2 en el subproblema P_3, y (3) generación del script de visualización y revisión de estilo del presente informe. Todo el código fuente, las demostraciones algebraicas y el análisis experimental fueron comprendidos, integrados y verificados por los tres integrantes del equipo.